

Main memory:

What is the meaning of physical memory and virtual memory?

Physical memory is the actual real memory used in RAM. Virtual memory as the name suggests is not real. The OS uses virtual memory as a memory management technique in which non-contiguous memory is presented to software as contiguous memory. If the RAM falls short of memory to accommodate more running processes, the OS allocates a portion of your hard drive to act as though it were RAM. That's what is referred to as virtual memory.

Physical memory is the only memory that is directly accessible to the CPU. CPU reads the instructions stored in the physical memory and executes them continuously. The data that is operated will also be stored in physical memory in uniform manner.

Virtual memory is one classification of memory which was created by using the hard disk for simulating additional RAM, the addressable space available for the user. Virtual addresses are mapped into real addresses.

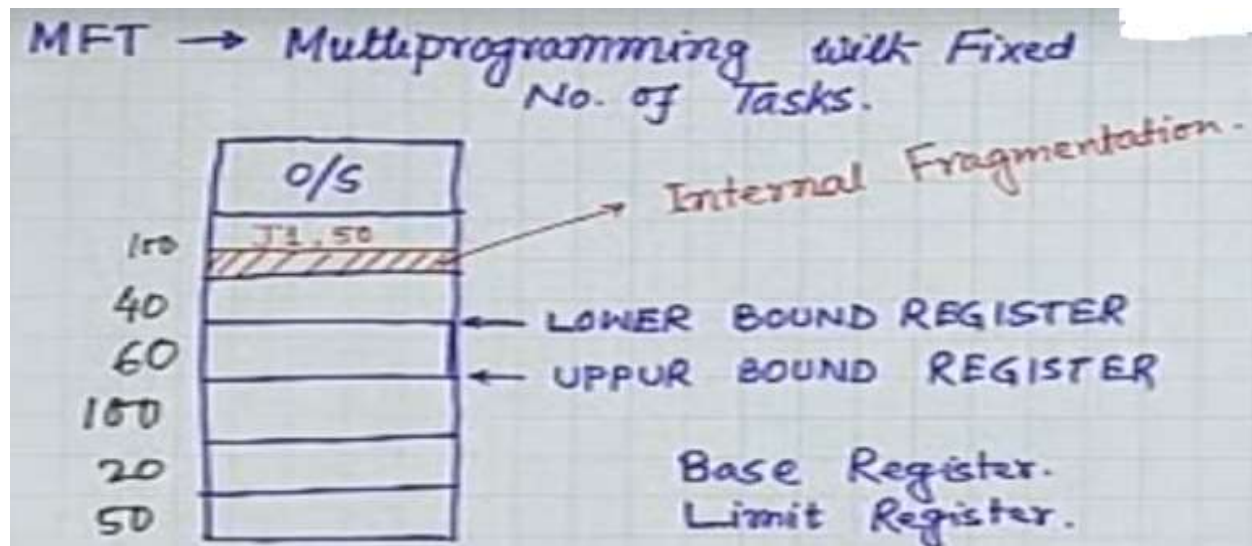
What is page and frame ??

In a memory management system that takes advantage of **paging**, the **OS** reads data from secondary storage in blocks called pages, all of which have identical size. The physical region of memory containing a single page is called a frame.

What is paging? Why paging is used?

OS performs an operation for storing and retrieving data from secondary storage devices for use in main memory. Paging is one of such memory management scheme. Data is retrieved from storage media by OS, in the same sized blocks called as pages. Paging allows the physical address space of the process to be non-contiguous. The whole program had to fit into storage contiguously.

Paging is to deal with external fragmentation problem. This is to allow the logical address space of a process to be noncontiguous, which makes the process to be allocated physical memory.



What is fragmentation in memory management?

As processes are loaded and removed from **memory**, the free **memory** space is broken into little pieces. It happens after sometimes that processes cannot be allocated to **memory** blocks considering their small size and **memory** blocks remains unused. This problem is known as **Fragmentation**.

Fragmentation occurs in a dynamic memory allocation system when most of the free blocks are too small to satisfy any request. It is generally termed as inability to use the available memory.

What is external fragmentation & internal fragmentation?

External fragmentation- Total memory space is enough to satisfy a request or to reside a process in it, but it is not contiguous so it cannot be used.

Internal fragmentation- Memory block assigned to process is bigger. Some portion of memory is left unused as it can not be used by another process.

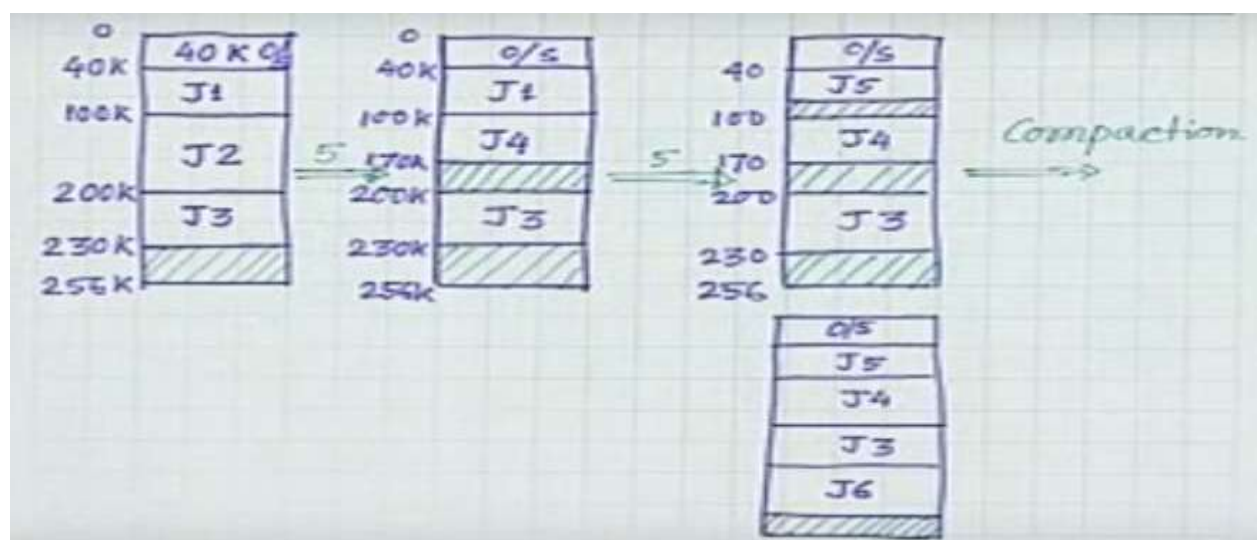
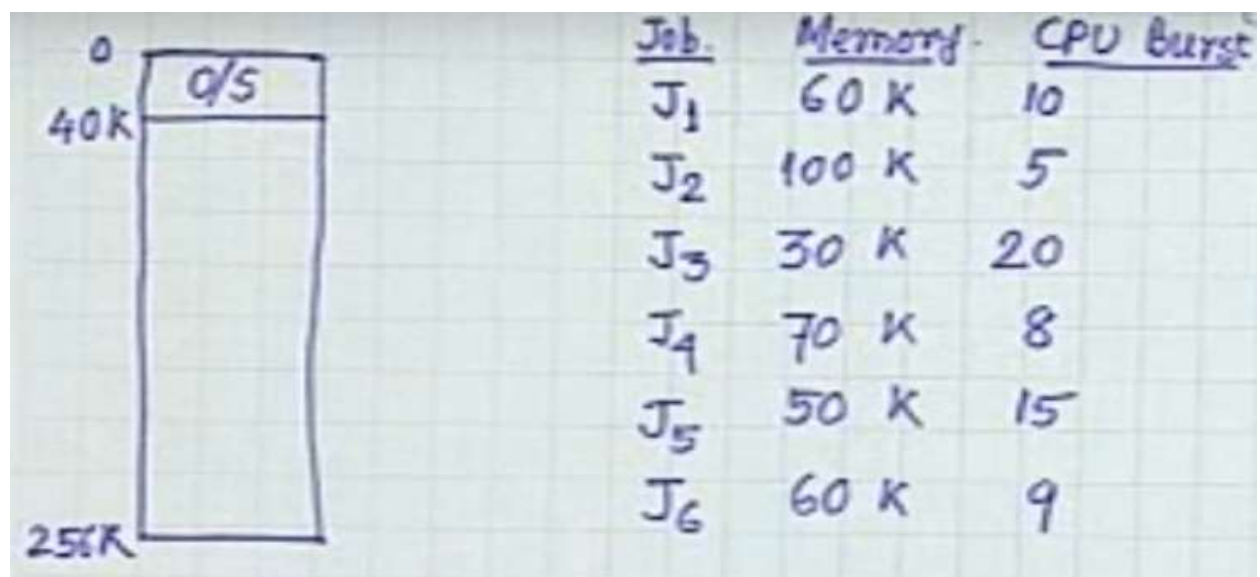
In such situation processes are loaded and removed from the memory. As a result of this, free holes exists to satisfy a request but is non contiguous i.e. the memory is fragmented into large number of small holes. This phenomenon is known as **External Fragmentation**.

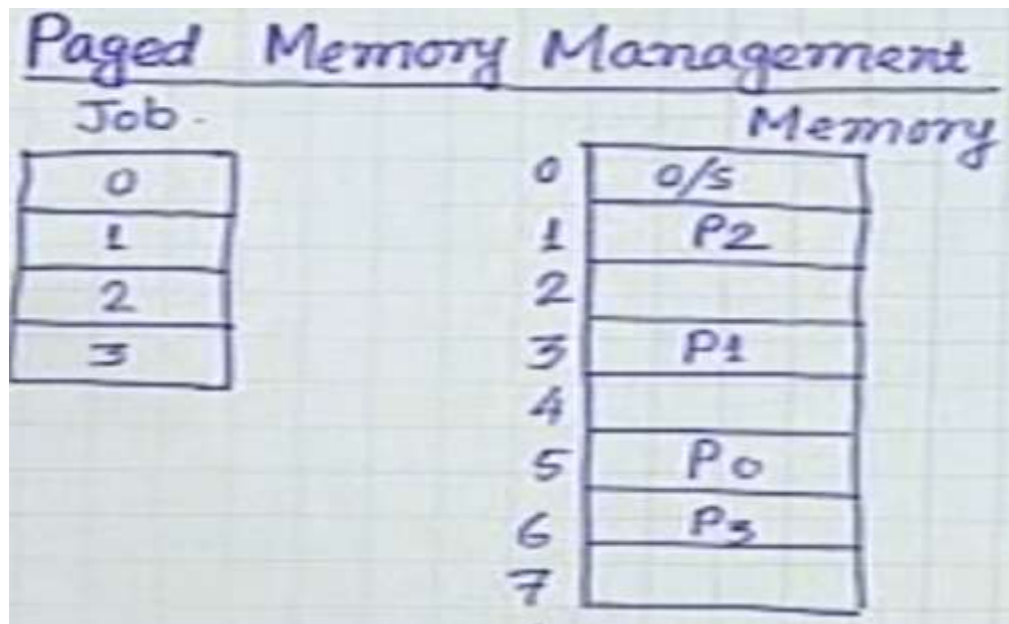
Also, at times the physical memory is broken into fixed size blocks and memory is allocated in unit of block sizes. The memory allocated to a space may be slightly larger than the requested memory. The difference between allocated and required memory is known as **Internal fragmentation** i.e. the memory that is internal to a partition but is of no use.

MFT: Internal Fragmentation.
External Fragmentation

LBR | $\Rightarrow$ Base Register
UBR | Limit Register.

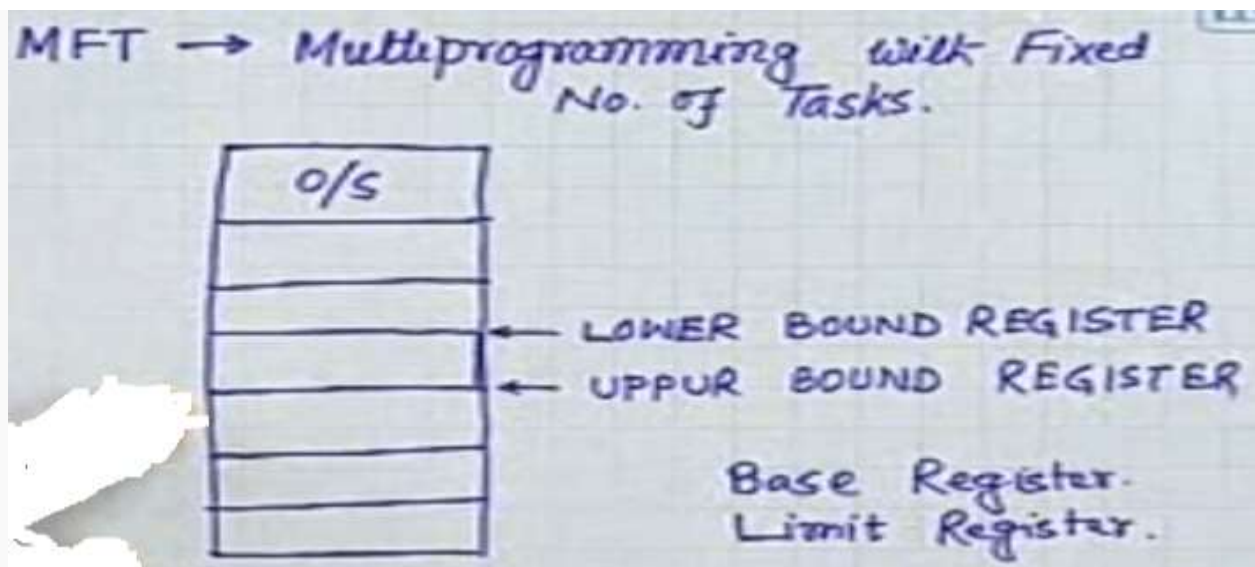
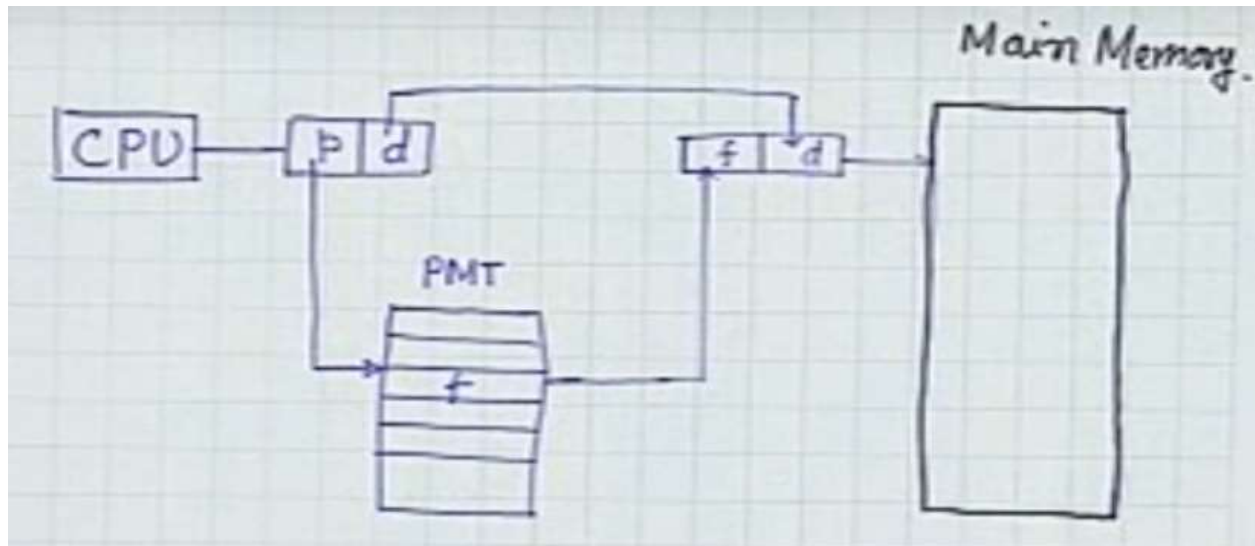
MVT: Multiprogramming With Variable number of tasks.





Logical address P₀=5, P₁=12, P₂=10, P₃=33; Page size P=10

P Page Size
 L Logical Address
 $L \Rightarrow p, d$
 $p = L \text{ div } P$
 $d = L \text{ mod } P$
 $\text{Ph. Addr.} = (f-1) * P + d$



Differentiate between logical and physical address.

Logical Vs physical address space

(1) An address generated by the CPU is commonly referred to as a logical address. The set of all logical addresses generated by a program is known as logical address space. Whereas, an address seen by the memory unit- that is, the one loaded into the memory-address register of the memory- is commonly referred to as physical address. The set of all physical addresses corresponding to the logical addresses is known as physical address space.

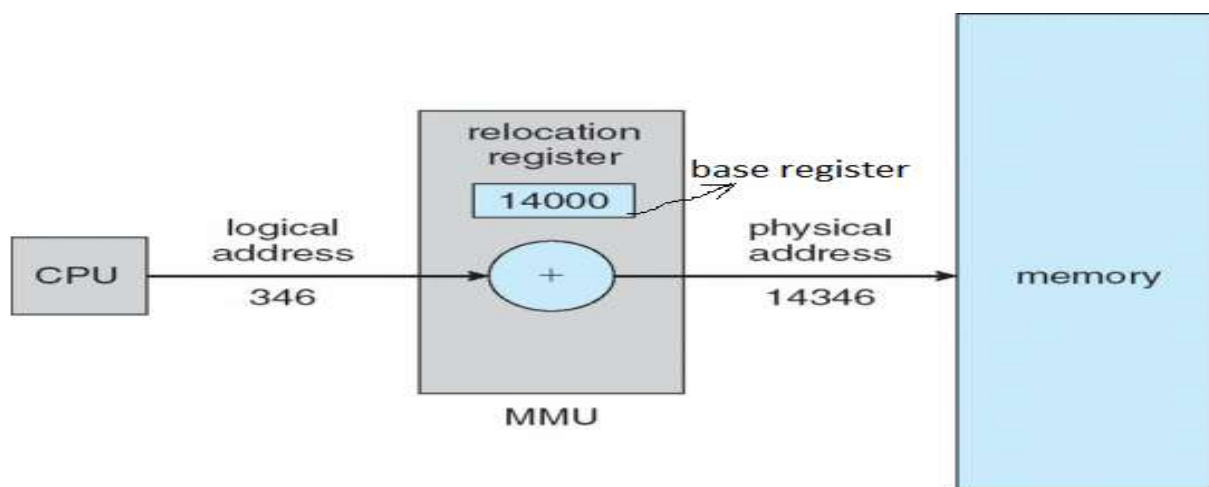
(2) The compile-time and load-time address-binding methods generate identical logical and physical addresses. However, in the execution-time address-binding scheme, the logical and physical-address spaces differ.

(3) The user program never sees the physical addresses. The program creates a pointer to a logical address, says 346, stores it in memory, manipulate it. The memory-mapping hardware device called the memory- management unit(MMU) converts logical addresses into physical addresses.

(4) Logical addresses range from 0 to max. User program that generates logical address thinks that the process runs in locations 0 to max.

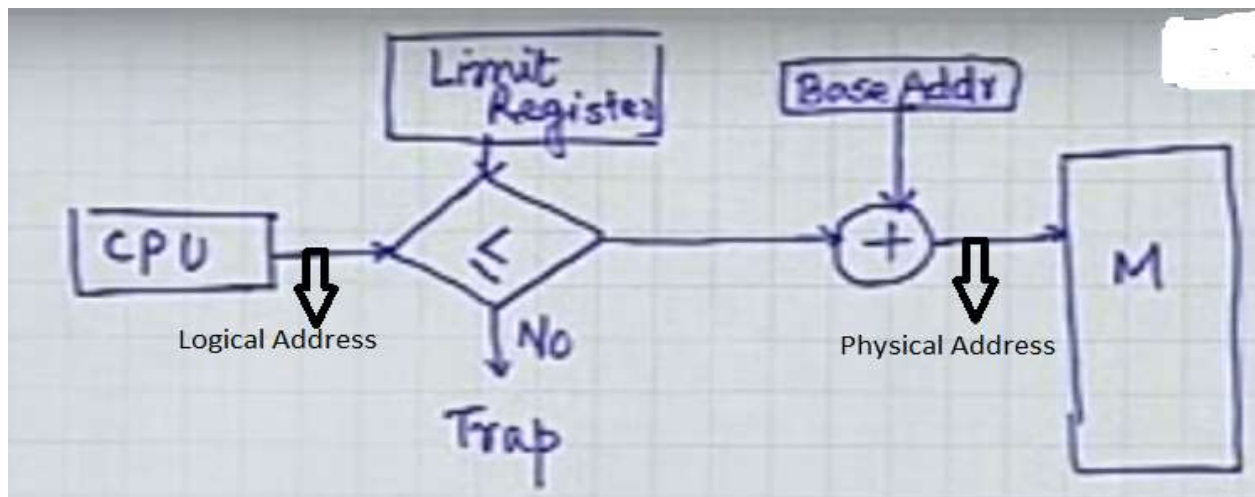
Logical addresses must be mapped to physical addresses before they are used. Physical addresses range from (R+0) to (R + max) for a base/relocation register value R.

(5) Example:



Mapping from logical to physical addresses using memory management unit (MMU) and relocation/base register

The value in relocation/base register is added to every logical address generated by a user process, at the time it is sent to memory, to generate corresponding physical address. In the above figure, base/ relocation value is 14000, and then an attempt by the user to access the location 346 is mapped to 14346.



Segmentation:

Intro

- Program is divided into parts known as segments
- These segments are variable in sizes
- Still, there's one limitation max segment size is defined
- Like Paging logical address consists two parts
 1. Segment no
 2. Offset

Segmentation

- It is similar to dynamic partitioning as variable size segments
- The difference is that, the segments need not be contiguous
- It eliminates internal fragmentation
- But like dynamic, it suffers from external fragmentation

Segmentation

- Paging was invisible but segmentation is visible to the user
- User or the assembler divides programs into various segments
- For modular programming purposes, it can be further divided into multiple segments
- Inconvenience is that the programmer must be aware of the max segment size

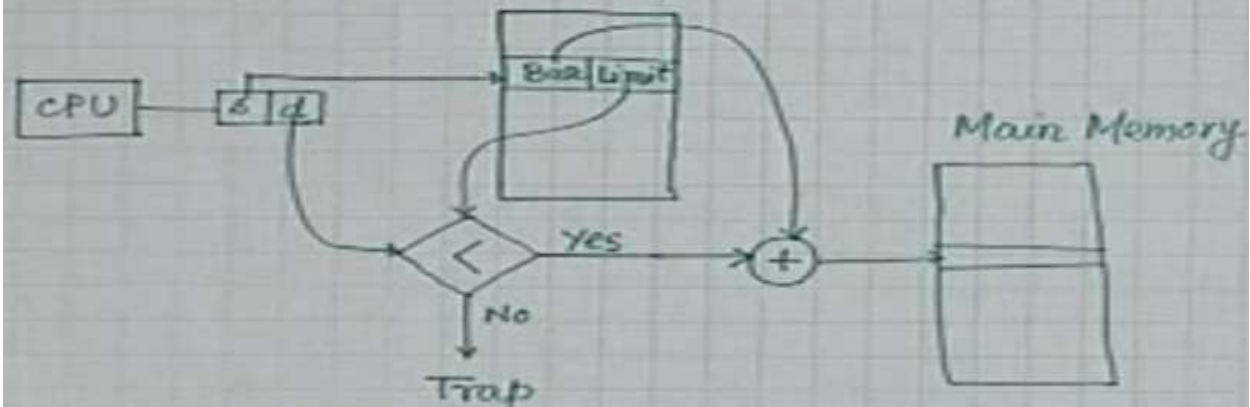
Segmentation

- There is no simple relationship between logical and physical address, which was pretty clear in paging
- Segment table keeps record of the segments as done by paging table before
- Logical address (segmentNo, Offset)

Next:

- Paging and segmentation both have separate advantages and disadvantages
- A scheme that combines these two is mostly used
- We will look into that scheme later while discussing various topics in virtual memory

Segmented Memory Mgmt.



segment No	limit	Base
0	1000	1400
1	400	6200
2	400	4300
3	1100	3200
4	1000	4700

find the physical address for the following logical addresses

1) 2, 53 2) 3, 352 3) 0, 1222

Step I:-

1400	Segment 0
2400	
3200	Segment 3
4300	Segment 2
4700	Segment 4
5700	
6300	Segment 1
6700	

Step II: $d \leq 1$

Seg 2:-

$$\begin{array}{r} 4300 \\ 53 \\ \hline 4353 \end{array}$$

Seg 3:-

$$\begin{array}{r} 3200 \\ 852 \\ \hline 4052 \end{array}$$

Seg 0:- illegal reference trap to 0-3

What is meant by contiguous memory allocation?

Contiguous memory allocation is a classical **memory allocation** model that assigns a process consecutive **memory** blocks (that is, **memory** blocks having consecutive

addresses). **Contiguous memory allocation** is one of the oldest **memory allocation** schemes. When a process needs to execute, **memory** is requested by the process.

Memory Allocation Scheme:

→ Fixed partition

→ Dynamic partition

Dynamic Storage Allocation Problem:

How to satisfy a request of size ***n*** from a list of free holes?

- n **First-fit:** Allocate the ***first*** hole that is big enough
- n **Best-fit:** Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
 - l Produces the smallest leftover hole
- n **Worst-fit:** Allocate the ***largest*** hole; must also search entire list
 - l Produces the largest leftover hole
- n First-fit and best-fit better than worst-fit in terms of speed and storage utilization

Q. Given 5 memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, 600 KB (in order).
 how would each of the first-fit, best-fit, worst-fit algorithms place processes of, 212 KB, 417 KB, 112 KB, 426 KB (in order)

First Fit

P1 212 KB ✓
 P2 417 KB ✓
 P3 112 KB ✓
 P4 426 KB

	100 KB
P1 (212)	500 KB
P3 (112)	200 KB
	300 KB
P2 (417)	600 KB

Best Fit

P1 212 KB ✓
 P2 417 KB ✓
 P3 112 KB ✓
 P4 426 KB

	100 KB
P2 (417)	500 KB
P3 (112)	200 KB
P1 (212)	300 KB
P4 (426)	600 KB

Worst Fit

P1 212 KB ✓
 P2 417 KB ✓
 P3 112 KB ✓
 P4 426 KB ✗

	100 KB
P2 (417)	500 KB
	200 KB
P3 (112)	300 KB
P1 (212)	600 KB

Physical Address Finding by page frame:

	Physical Address	Physical Address	Physical Address
page 0	0	0	0
page 1	1	1	1
page 2	2	2	2
page 3	3	3	3
page 4	4	4	4
page 5	5	5	5
page 6	6	6	6
page 7	7	7	7
page 8	8	8	8
page 9	9	9	9
page 10	10	10	10
page 11	11	11	11
page 12	12	12	12
page 13	13	13	13
page 14	14	14	14
page 15	15	15	15
page 16	16	16	16
page 17	17	17	17
page 18	18	18	18
page 19	19	19	19
page 20	20	20	20
page 21	21	21	21
page 22	22	22	22
page 23	23	23	23
page 24	24	24	24
page 25	25	25	25
page 26	26	26	26
page 27	27	27	27
page 28	28	28	28
page 29	29	29	29
page 30	30	30	30
page 31	31	31	31
page 32	32	32	32
page 33	33	33	33
page 34	34	34	34
page 35	35	35	35
page 36	36	36	36
page 37	37	37	37
page 38	38	38	38
page 39	39	39	39
page 40	40	40	40
page 41	41	41	41
page 42	42	42	42
page 43	43	43	43
page 44	44	44	44
page 45	45	45	45
page 46	46	46	46
page 47	47	47	47
page 48	48	48	48
page 49	49	49	49
page 50	50	50	50
page 51	51	51	51
page 52	52	52	52
page 53	53	53	53
page 54	54	54	54
page 55	55	55	55
page 56	56	56	56
page 57	57	57	57
page 58	58	58	58
page 59	59	59	59
page 60	60	60	60
page 61	61	61	61
page 62	62	62	62
page 63	63	63	63
page 64	64	64	64
page 65	65	65	65
page 66	66	66	66
page 67	67	67	67
page 68	68	68	68
page 69	69	69	69
page 70	70	70	70
page 71	71	71	71
page 72	72	72	72
page 73	73	73	73
page 74	74	74	74
page 75	75	75	75
page 76	76	76	76
page 77	77	77	77
page 78	78	78	78
page 79	79	79	79
page 80	80	80	80
page 81	81	81	81
page 82	82	82	82
page 83	83	83	83
page 84	84	84	84
page 85	85	85	85
page 86	86	86	86
page 87	87	87	87
page 88	88	88	88
page 89	89	89	89
page 90	90	90	90
page 91	91	91	91
page 92	92	92	92
page 93	93	93	93
page 94	94	94	94
page 95	95	95	95
page 96	96	96	96
page 97	97	97	97
page 98	98	98	98
page 99	99	99	99

0	0	
1	4	
2	8	
3	12	
4	16	
5	20	
6	24	
7	28	
8	32	
9	36	
10	40	
11	44	
12	48	
13	52	
14	56	
15	60	
16	64	
17	68	
18	72	
19	76	
20	80	
21	84	
22	88	
23	92	
24	96	
25	100	
26	104	
27	108	
28	112	
29	116	
30	120	
31	124	
32	128	
33	132	
34	136	
35	140	
36	144	
37	148	
38	152	
39	156	
40	160	
41	164	
42	168	
43	172	
44	176	
45	180	
46	184	
47	188	
48	192	
49	196	
50	200	
51	204	
52	208	
53	212	
54	216	
55	220	
56	224	
57	228	
58	232	
59	236	
60	240	
61	244	
62	248	
63	252	
64	256	
65	260	
66	264	
67	268	
68	272	
69	276	
70	280	
71	284	
72	288	
73	292	
74	296	
75	300	
76	304	
77	308	
78	312	
79	316	
80	320	
81	324	
82	328	
83	332	
84	336	
85	340	
86	344	
87	348	
88	352	
89	356	
90	360	
91	364	
92	368	
93	372	
94	376	
95	380	
96	384	
97	388	
98	392	
99	396	

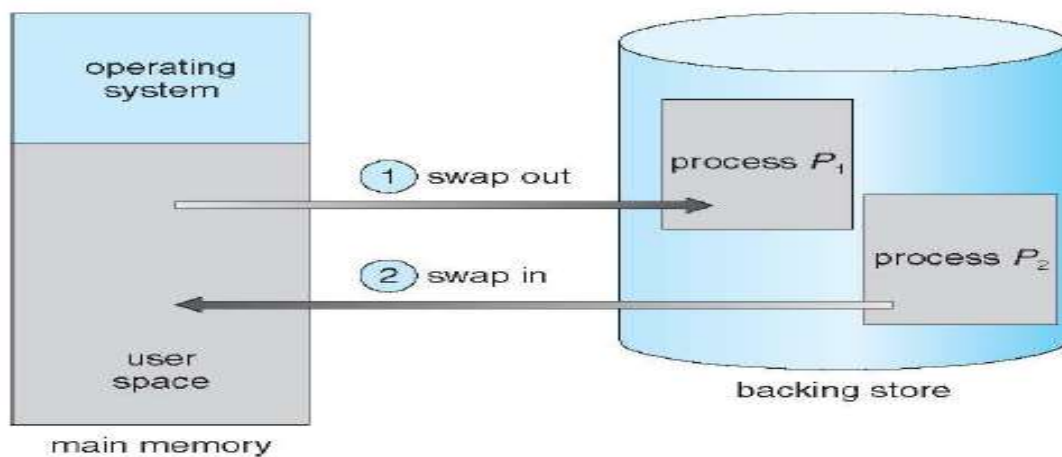
$$\begin{aligned}
 \text{page size} &= 4B \\
 P \cdot A &= (S \cdot N * P \cdot S) + \text{offset} \\
 &= (7 \times 4) + 0 = 28 \\
 P \cdot A &= (2 \times 4) + 1 = 8 + 1 = 9
 \end{aligned}$$

What is swap on a computer?

A **swap** file (or **swap** space or, in Windows NT, a pagefile) is a space on a hard disk used as the virtual memory extension of a **computer's** real memory (RAM). Having a **swap** file allows your **computer's** operating system to pretend that you have more RAM than you actually do.

To replace pages or segments of data in memory. **Swapping** is a useful technique that enables a computer to execute programs and manipulate data files larger than main memory. The **operating system** copies as much data as possible into main memory, and leaves the rest on the disk.

Schematic View of Swapping



What is address bind?

Address binding is the process of mapping the program's logical or virtual **addresses** to corresponding physical or main memory **addresses**. In other words, a given logical **address** is mapped by the MMU (Memory Management Unit) to a physical **address**.